

Unit-2: Introduction to Deep Learning

Mr.HIMANSHU TIWARI

Deep Learning with NLP(303105479)

Artificial Intelligence and Data Science Department

Content

- Introduction to the Deep Learning
- Basic Structure of Neural Network
- Types of Activation Function
- Types of Neural Network
 - Feedforward Neural Network
 - Convolutional Neural Network
 - Recurrent Neural Network
 - Encoder Decoder Networks
 - Recursive Neural Network
 - Multilayer Neural Network
- Stochastic Gradient Descent
- Backpropagation Algorithm

Introduction:

- Deep Learning, a branch of machine learning, revolves around the utilization of algorithms known as artificial neural networks, which are inspired by the structure and function of the brain.
- It uses artificial neural networks with multiple layers to automatically learn patterns from data. This enables accurate predictions and complex tasks like image recognition.
- Deep learning has made significant advancements and has applications in various fields, driving technological innovation.

- Deep learning is a machine learning branch that relies on artificial neural networks (ANNs) to learn intricate patterns and relationships in data.
- **Unlike traditional programming, deep learning algorithms do not require explicit programming for every task.**
- It has gained popularity due to advancements in processing power and access to extensive datasets.

- In deep learning, neural networks are designed to mimic the structure and functionality of the human brain and are composed of interconnected nodes organized into layers. The nodes process and manipulate data, enabling the network to learn and make sense of complex patterns.
- Deep Learning is distinguished by its utilization of deep neural networks, which consist of multiple interconnected layers of nodes. These networks excel at learning complex data representations by uncovering hierarchical patterns and features. One of the key advantages of Deep Learning is its ability to automatically learn and improve from data without the need for manual feature engineering.

- Deep Learning has seen remarkable accomplishments across diverse domains like image recognition, natural language processing, speech recognition, and recommendation systems. Key architectures include Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Deep Belief Networks (DBNs).
- Training deep neural networks traditionally demands substantial data and computational resources. Nonetheless, the emergence of cloud computing and specialized hardware, like Graphics Processing Units (GPUs), has facilitated the training process.

Example of Deep Learning^[1]

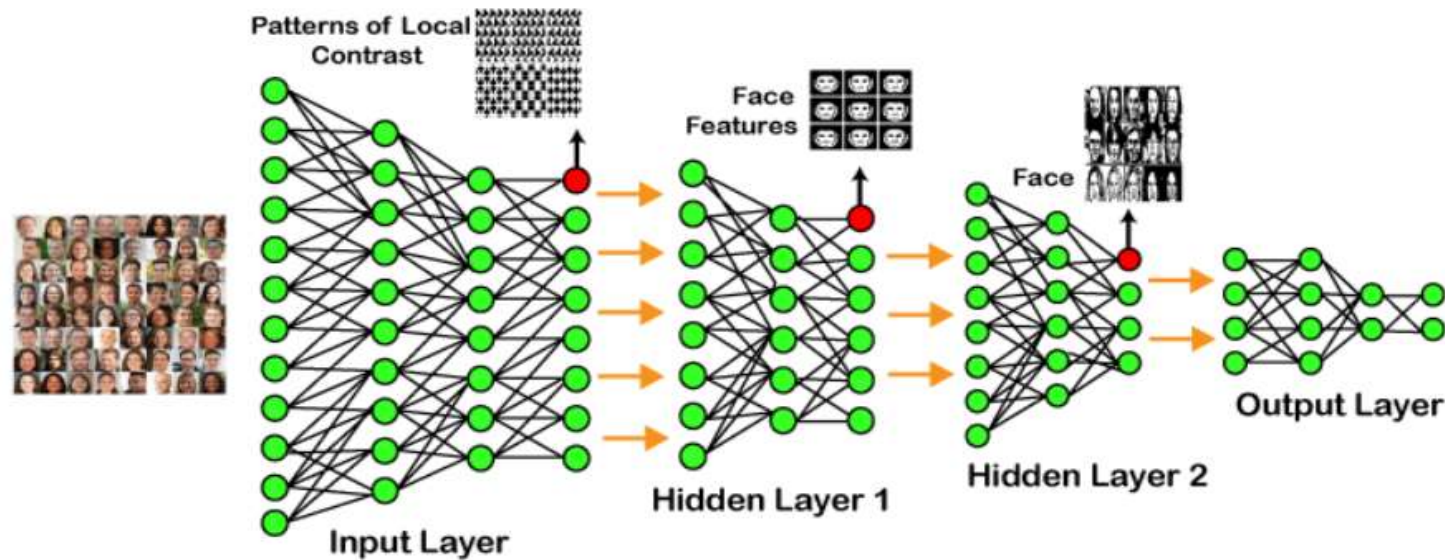


Fig 2.1. Example of Deep Learning

- In the provided example, the raw image data is fed into the initial input layer. This layer then analyzes the local contrast patterns, distinguishing between colors, luminosity, and other characteristics.
- Subsequently, the first hidden layer identifies facial features such as eyes, nose, and lips, focusing on these specific elements. The hidden layer then aligns these facial features with the appropriate face template.
- The second hidden layer plays a crucial role in determining the correct face, as demonstrated in the image above. The processed information is then passed to the output layer. To address more intricate tasks, additional hidden layers can be added.
- For instance, if the objective is to identify faces with specific attributes like a fair or dark complexion.
- By increasing the number of hidden layers, complex problems can be effectively tackled.

Types of Learnings

- Deep learning can be used for supervised, unsupervised as well as reinforcement machine learning.
- **Supervised Machine Learning** trains a neural network using labeled datasets to make predictions or classify data.
- The network learns from input features and target variables, reducing prediction errors through backpropagation.
- Deep learning algorithms like CNNs and RNNs are frequently employed for supervised tasks such as image classification, sentiment analysis, and language translation.

- **Unsupervised Machine Learning** involves training a neural network to identify patterns or cluster data without the use of labeled datasets or target variables.
- The network independently discovers hidden patterns or relationships within the data.
- Deep learning algorithms like autoencoders and generative models are commonly utilized for unsupervised tasks such as clustering, dimensionality reduction, and anomaly detection.

- **Reinforcement Machine Learning** trains agents to make decisions that maximize rewards in an environment.
- Through interaction with the environment, agents learn from observed rewards.
- Deep learning is utilized to develop policies that optimize cumulative rewards over time.
- Deep reinforcement learning algorithms, including Deep Q networks and Deep Deterministic Policy Gradient (DDPG), are widely employed in domains like robotics and game playing.

applications of Deep Learning

- Image and video recognition
- Generative models
- Autonomous vehicles
- Image classification
- Speech recognition
- Natural Language Processing
- Recommender systems
- Fraud detection
- Game-playing AI
- Time series forecasting

Advantages & Disadvantages of Deep Learning

Advantages:

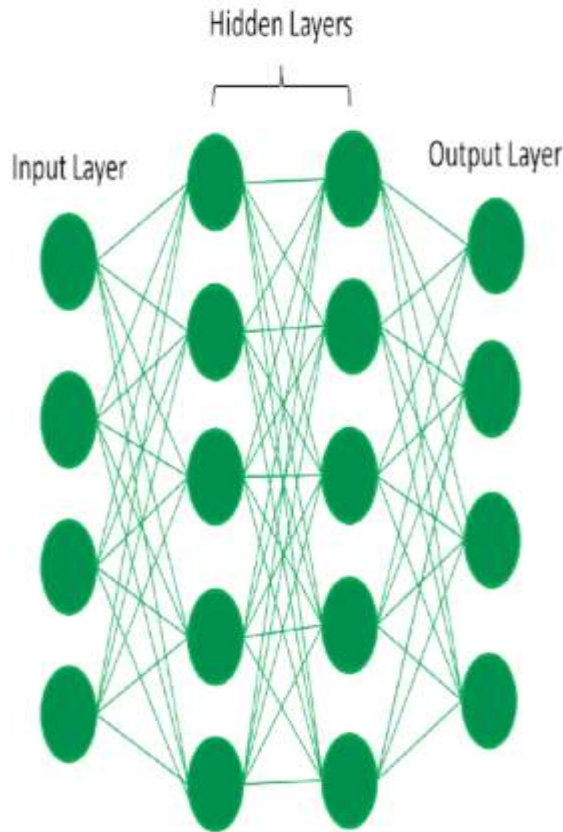
- High accuracy
- Automated feature engineering
- Scalability
- Flexibility
- Continual improvement

Disadvantages:

- High computational requirements
- Requires large amounts of labeled data
- Interpretability
- Black-box nature
- Continual improvement

Basic Structure of Neural Network

- Artificial neural networks, also referred to as neural networks or neural nets, are designed based on the principles governing the structure and functioning of human neurons.
- In this architecture, an artificial neural network consists of multiple layers. The initial layer, known as the input layer, receives input from external sources and transmits it to the subsequent layer, known as the hidden layer.
- Each neuron within the hidden layer receives information from neurons in the preceding layer, calculates the weighted sum of inputs, and transfers the result to neurons in the subsequent layer.
- The connections between neurons are assigned weights, which determine the relative influence of inputs from the previous layer. These weights are adjusted during the training process to optimize the performance of the model and improve its accuracy.



Fully Connected Artificial Neural Network

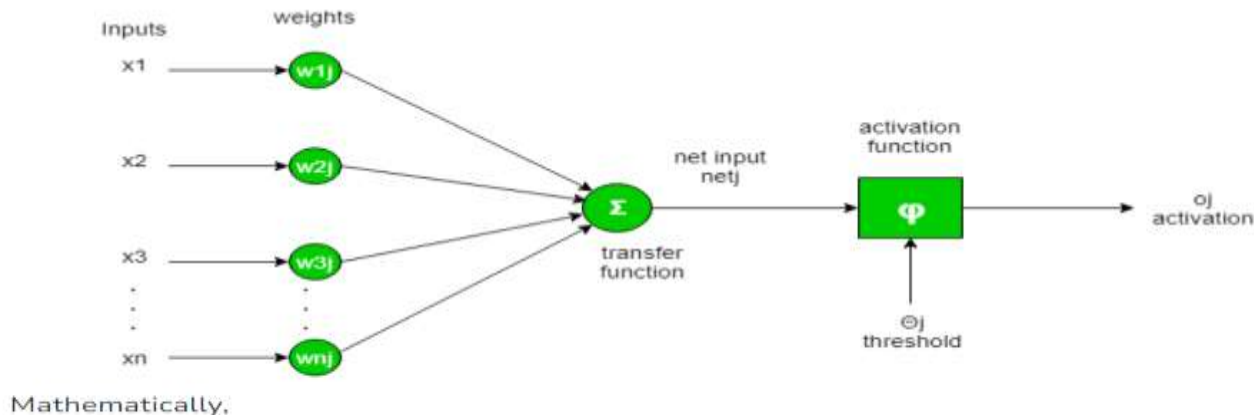
- Artificial neurons, referred to as units, are integral components of artificial neural networks. These networks consist of interconnected artificial neurons organized into layers.
- The complexity of a neural network is determined by the intricate patterns present in the dataset, whether a layer contains a small number or millions of units.
- Typically, an artificial neural network comprises an input layer, an output layer, and hidden layers. The input layer receives data from the external environment that the neural network aims to analyze or learn from.

Difference between Machine Learning and Deep Learning^[1]

Machine Learning	Deep Learning
Apply statistical algorithms to learn the hidden patterns and relationships in the dataset.	Uses artificial neural network architecture to learn the hidden patterns and relationships in the dataset.
Can work on the smaller amount of dataset	Requires the larger volume of dataset compared to machine learning
Better for the low-label task.	Better for complex task like image processing, natural language processing, etc.
Takes less time to train the model	Takes more time to train the model
A model is created by relevant features which are manually extracted from images to detect an object in the image.	Relevant features are automatically extracted from images. It is an end-to-end learning process.
Less complex and easy to interpret the result.	More complex, it works like the black box interpretations of the result are not easy.

Activation Functions

an artificial neuron computes the total value of its inputs, taking into account their individual weights, and includes an additional adjustment called bias. This process is represented by the net input in the accompanying figure.



$$\text{net input} = \sum (\text{weight} * \text{input}) + \text{bias}$$

- The net input of an artificial neuron can vary widely, ranging from negative infinity to positive infinity. Since the neuron doesn't inherently know how to limit this value, it cannot determine its firing pattern on its own.
- This is where the activation function plays a crucial role in an artificial neural network.
- The activation function determines whether a neuron should be activated or not by constraining the value of the net input within a specific range.
- By applying a non-linear transformation to the input, the activation function shapes and regulates the neuron's output before passing it to the next layer of neurons or as the final output of the network.

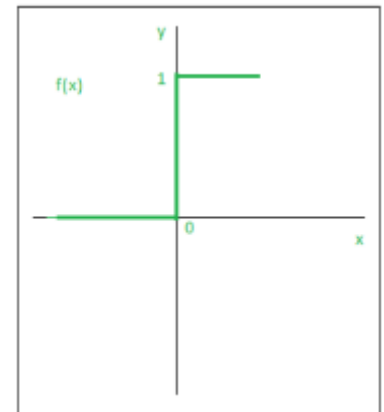
Several different types of activation functions are used in Deep Learning. Some of them are explained below:

1. Step Function: The Step Function is a straightforward type of activation function, known for its simplicity. It operates by comparing the net input value, denoted as "y," with a specified threshold. If the net input exceeds the threshold, the neuron is activated.

Mathematically, the Step Function can be represented as follows:

$$\begin{aligned} f(x) &= 1, \text{ if } x \geq 0 \\ f(x) &= 0, \text{ if } x < 0 \end{aligned}$$

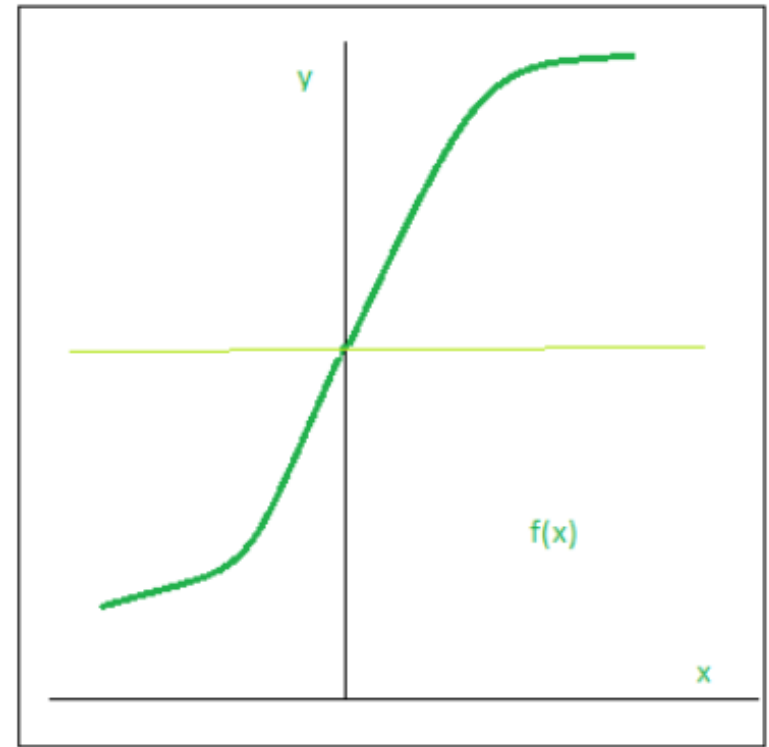
Side figure shows the graphical representation of step function



2. Sigmoid Function:

Sigmoid function is a widely used activation function. It is defined as:

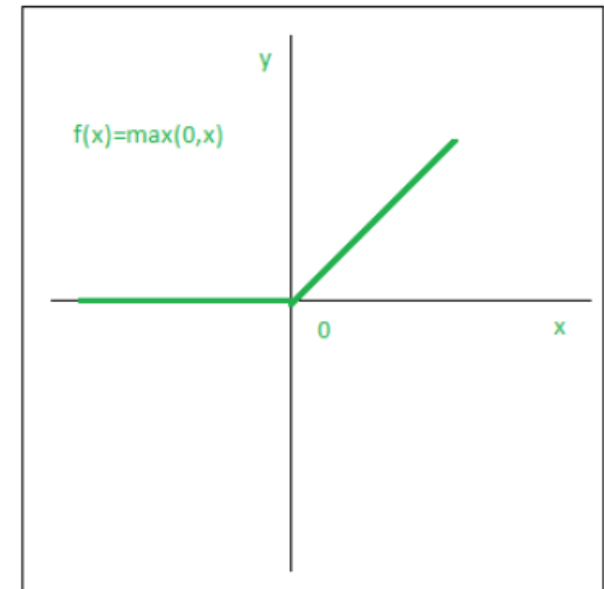
- The sigmoid function is a smooth and continuously differentiable function. Its key advantage over the step and linear functions is its non-linearity, which makes it an intriguing feature.
- When multiple neurons adopt the sigmoid function as their activation function, the resulting output is also non-linear. The sigmoid function has a distinct S-shaped curve and maps input values to a range between 0 and 1.



3. ReLU:

The ReLU function is the Rectified linear unit. It is the most widely used activation function. It is defined as:

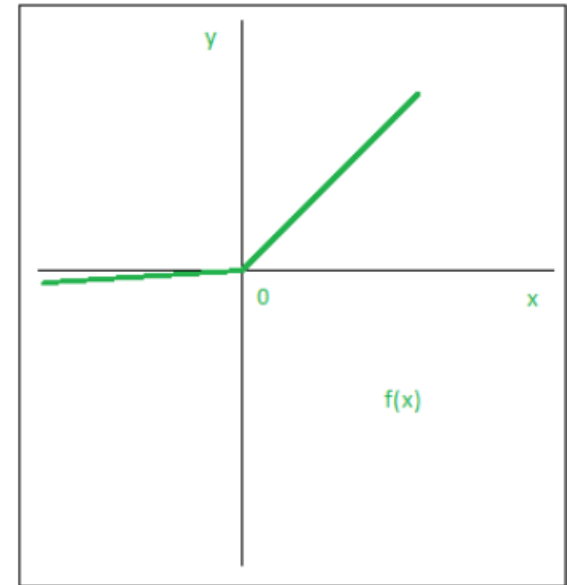
- The primary benefit of employing the Rectified Linear Unit (ReLU) function as an activation function is that it does not activate all neurons simultaneously.
- In practical terms, this means that when the input to the ReLU function is negative, it is converted to zero, resulting in the neuron remaining inactive or "not activated."



4. LeakyReLU:

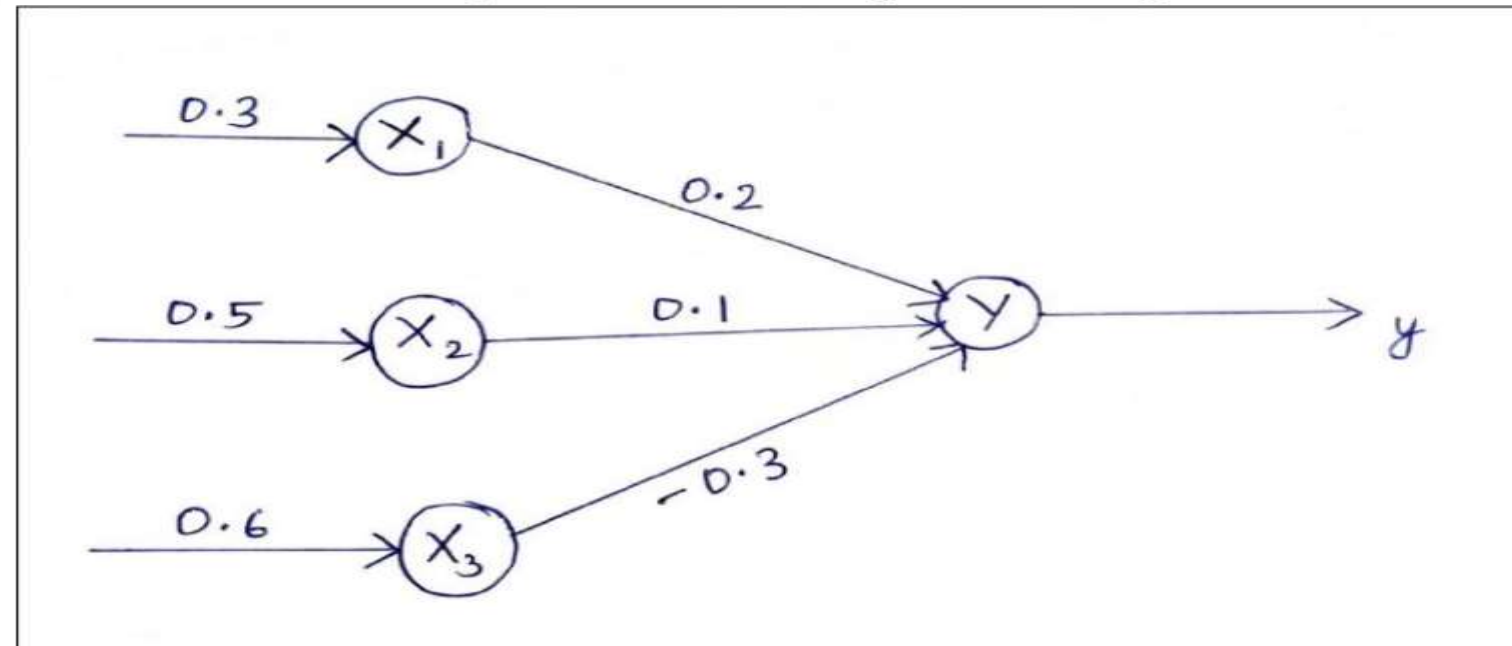
Leaky ReLU function is nothing but an improved version of the ReLU function. Instead of defining the ReLU function as 0 for x less than 0, we define it as a small linear component of x . It can be defined as:

$$\begin{aligned} f(x) &= ax, x < 0 \\ f(x) &= x, \text{ otherwise} \end{aligned}$$



Activation Example

(1) For the network shown, calculate the net input to the output neuron.



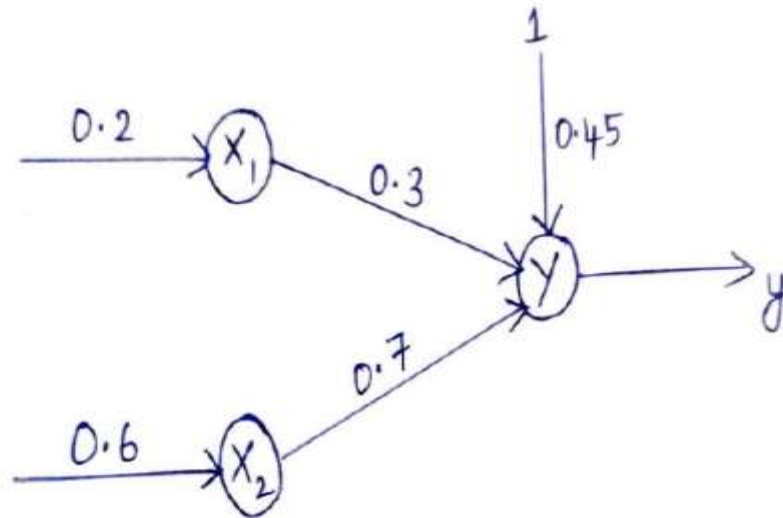
Inputs,	$x_1=0.3$	$x_2=0.5$	$x_3=0.6$
Weights,	$W_1=0.2$	$W_2=0.1$	$W_3=-0.3$

Activation Example

Net Input,

$$\begin{aligned} y_{in} &= x_1 W_1 + x_2 W_2 + x_3 W_3 \\ &= 0.3 * 0.2 + 0.5 * 0.1 + 0.6 * -0.3 \\ &= 0.06 + 0.05 - 0.18 \\ &= 0.11 - 0.18 \\ &= -0.07 \end{aligned}$$

(2) Calculate the net input for the network with bias included.



Inputs, $x_1=0.2$ $x_2=0.6$

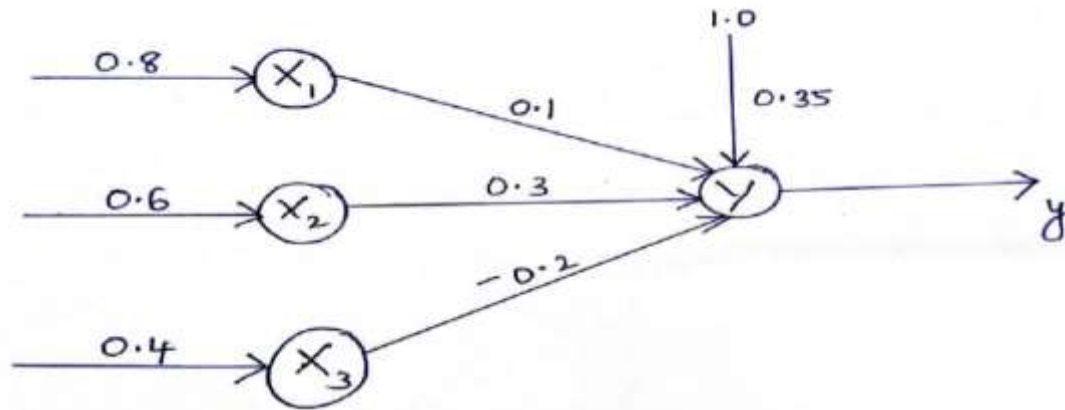
Weights, $W_1=0.3$ $W_2=0.7$

Net Input,

$$\begin{aligned} y_{in} &= \text{bias} + x_1 W_1 + x_2 W_2 \\ &= 0.45 + 0.2 \times 0.3 + 0.6 \times 0.7 \\ &= 0.45 + 0.06 + 0.42 \\ &= 0.93 \end{aligned}$$

Obtain the output of the neuron Y for the network shown using activation function.

(i) binary sigmoidal (ii) bipolar sigmoidal.



$$\begin{aligned}\text{Net Input, } y_{in} &= b + x_1W_1 + x_2W_2 + x_3W_3 \\ &= 0.35 + 0.8*0.1 + 0.6*0.3 + 0.4*-0.2 \\ &= 0.35 + 0.08 + 0.18 - 0.08 \\ &= 0.61 - 0.08 \\ &= 0.53\end{aligned}$$

(i) Binary Sigmoidal Activation Function

$$\begin{aligned}y &= f(y_{in}) \\ &= \frac{1}{1+e^{-y_{in}}} \\ &= \frac{1}{1+e^{-0.53}} = 0.625\end{aligned}$$

(ii) Bipolar Sigmoidal Activation Function

$$\begin{aligned}y &= f(y_{in}) \\ &= \frac{1+e^{-y_{in}}}{2} - 1 \\ &= \frac{1+e^{-0.53}}{2} - 1 = 0.259\end{aligned}$$

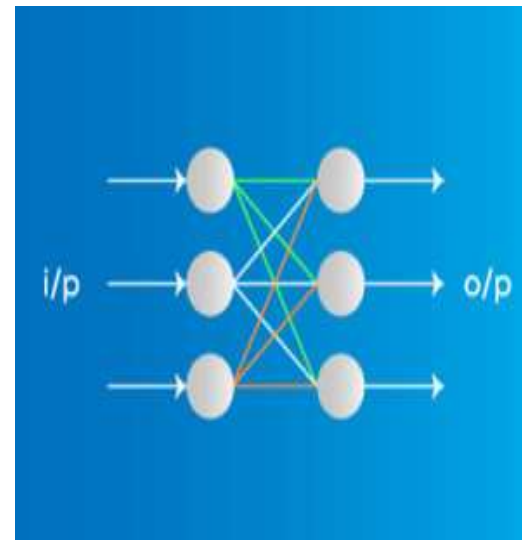
Types of neural networks

- Deep learning models possess the capability to autonomously extract features from data, rendering them highly suitable for tasks like image recognition, speech recognition, and natural language processing.
- The prominent architectures employed in deep learning include following types of Neural Networks:
 1. feedforward neural networks
 2. convolutional neural networks (CNNs)
 3. recurrent neural networks (RNNs)
 4. Encoder Decoder Networks
 5. Recursive Neural Network
 6. Multilayer Neural Network

Types of neural networks

1. feedforward neural network

- A basic type of neural network is characterized by the unidirectional flow of input data through artificial neural nodes, ultimately exiting through output nodes.
- These networks may or may not include hidden layers, but always consist of input and output layers.
- Consequently, they can be categorized as either single-layered or multi-layered feed-forward neural networks.



2.7 feedforward neural network

Types of neural networks

- The number of layers in a feed-forward neural network depends on the complexity of the function it aims to learn.
- These networks employ unidirectional forward propagation, meaning information flows only from input to output without any backward propagation. The weights in the network remain static and do not update during training.
- In this type of network, inputs are multiplied by the weights and passed through an activation function. Common choices for activation functions include the classifying activation function or the step activation function.
- For instance, in the step activation function, a neuron is activated and outputs a value of 1 if it exceeds a certain threshold (typically 0). Conversely, if the neuron falls below the threshold, it is not activated and produces a value of -1.
- Feed-forward networks are relatively easy to maintain and are well-suited for handling noisy data. They can effectively process data with various levels of noise and provide valuable insights.

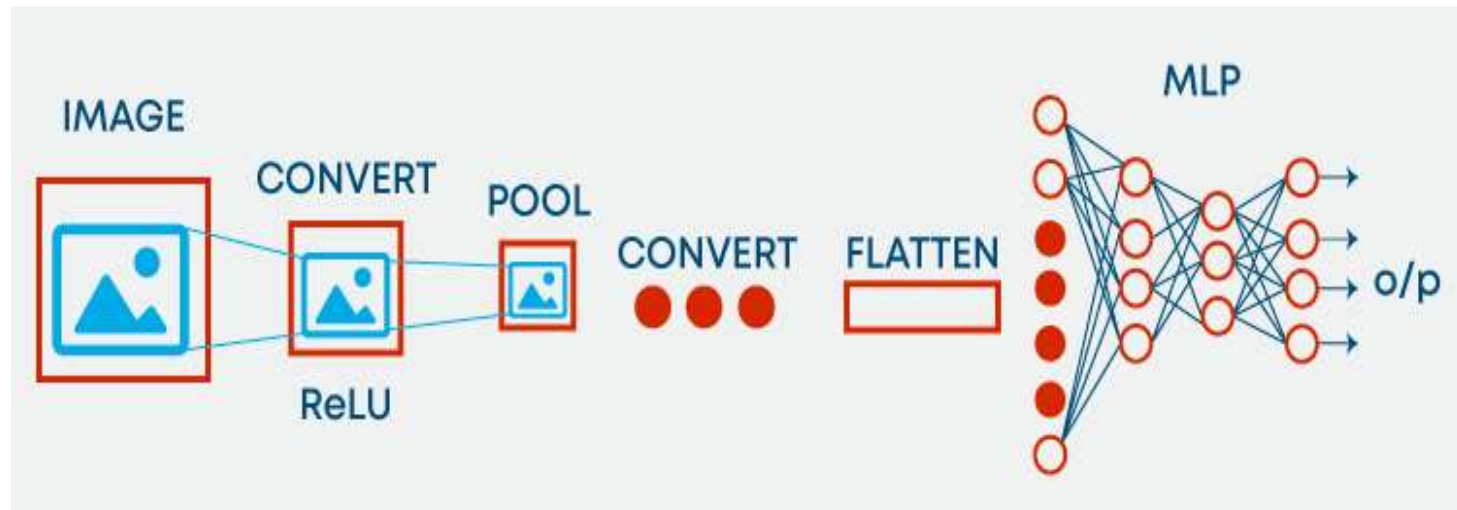
Types of neural networks

Applications on Feed Forward Neural Networks:

1. Simple classification (where traditional Machine-learning based classification algorithms have limitations)
2. Face recognition [Simple straight forward image processing]
3. Computer vision [Where target classes are difficult to classify]
4. Speech Recognition

Types of neural networks

Convolutional Neural Network

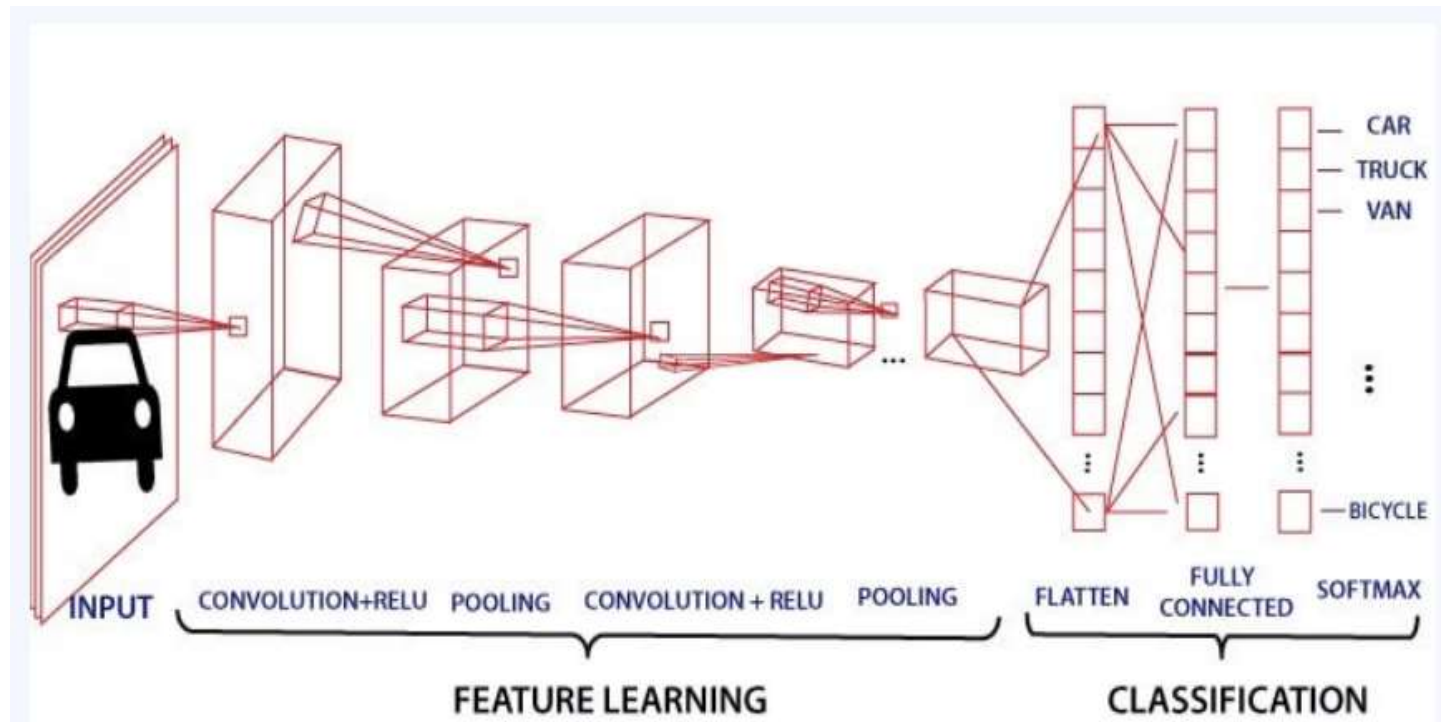


2.7 Convolutional neural network

Types of neural networks

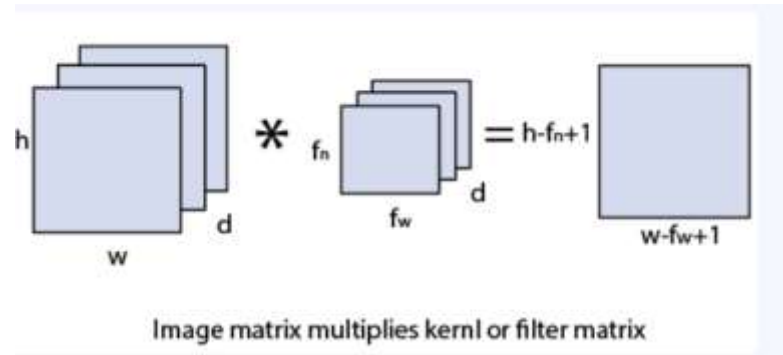
- A convolutional neural network (CNN) differs from the conventional two-dimensional neural network by employing a three-dimensional arrangement of neurons.
- The initial layer in a CNN is known as the convolutional layer, where each neuron processes information from a small portion of the visual field.
- Input features are handled in batches, similar to applying a filter. By breaking down images into parts, the network can repeatedly compute these operations to complete the overall image processing.
- This involves converting the image from RGB or HSI scale to grayscale, and manipulating pixel values to detect edges and classify images into different categories.
- Propagation in a CNN is unidirectional, wherein one or more convolutional layers are followed by pooling.
- Alternatively, it can be bidirectional, where the output of the convolutional layer is passed to a fully connected neural network for image classification, as depicted in the diagram above. Filters are utilized to extract specific components of the image.

Types of neural networks



Types of neural networks

The Convolution Layers are the initial layers to pull out features from the image. It maintains the relationship between pixels by learning features using a small input data sequence. It is a mathematical term that takes two inputs, an image matrix and a kernel or filter. The result is calculated by:



Types of neural networks

Now, let us take an example and solve a 5x5 image matrix whose pixel values are 0, 1 and the filter matrix as 3x3:

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

5 × 5 – Image Matrix

$$\times \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

3 × 3 – Filter Matrix

1	1	1	0	0
0	1	1	1	0
0	0	1 _{x1}	1 _{x0}	1 _{x1}
0	0	1 _{x0}	1 _{x1}	0 _{x0}
0	1	1 _{x1}	0 _{x0}	0 _{x1}

Image

4	3	4
2	4	3
2	3	4

Convolved
Feature

- The convolution of the image with different filter values can produce a blur or sharpened image.
- The size of the output image is calculated by:

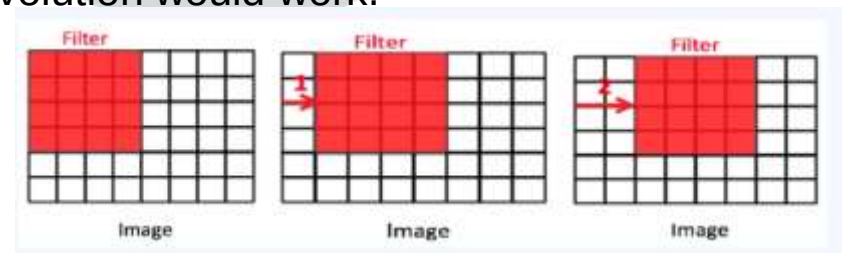
$$i-k+1 = 5-3+1=3$$

Types of neural networks

Strides:

- When the array is created, the pixels are shifted over to the input matrix. The number of pixels turning to the input matrix is known as the strides.
- When the number of strides is 1, we move the filters to 1 pixel at a time.
- Similarly, when the number of strides is 2, we carry the filters to 2 pixels, and so on.
- They are essential because they control the convolution of the filter against the input, i.e., Strides are responsible for regulating the features that could be missed while flattening the image.
- They denote the number of steps we are moving in each convolution. The following figure shows how the convolution would work.

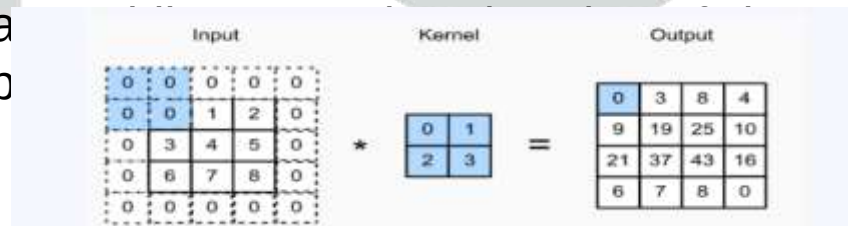
$$O = [i - k/s] + 1 = [5 - 3/2] + 1 = 2$$



Types of neural networks

Padding :

- The padding plays a vital role in creating CNN. After the convolution operation, the original size of the image is shrunk. Also, in the image classification task, there are multiple convolution layers after which our original image is shrunk after every step, which we don't want.
- When the kernel moves over the original image, it passes through the middle layer more times than the edge layers, due to which there occurs an overlap.
- To overcome this problem, a new concept was introduced named padding. It is an additional layer that can add to the borders of an image picture. For example,



Pooling:

- The pooling layer is another building block of a CNN and plays a vital role in pre-processing an image.
- In the pre-process, the image size shrinks by reducing the number of parameters if the image is too large.
- When the picture is shrunk, the pixel density is also reduced, the downscaled image is obtained from the previous layers.
- Basically, **its function is to progressively reduce the spatial size of the image to reduce the network complexity and computational cost.**
- Spatial pooling is also known as **downsampling** or **subsampling** that reduces the dimensionality of each map but retains the essential features.
- A rectified linear activation function, or ReLU, is applied to each value in the feature map.
- Relu is a simple and effective nonlinearity that does not change the values in the feature map but is present because later subsequent pooling layers are added. Pooling is added after the nonlinearity is applied to the feature maps.

Max Pooling

- Max pooling is a rule to take the maximum of a region and help to proceed with the most crucial features from the image.
- It is a sample-based process that transfers continuous functions into discrete counterparts.
- Its primary objective is to downscale an input by reducing its dimensionality and contained in the sub-region that were rejected.

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4

$$\text{Max}([4, 3, 1, 3]) = 4$$

4	3	1	5
1	3	4	8
4	5	4	3
6	5	9	4

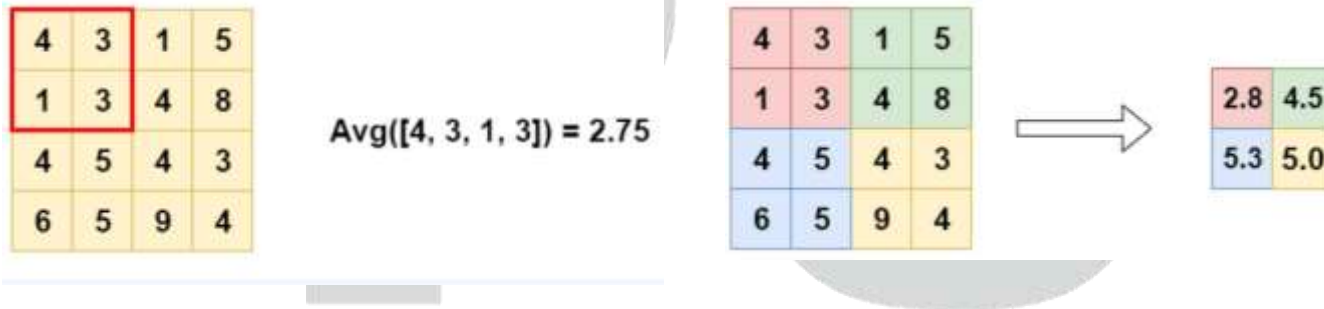


4	8
6	9

Types of neural networks

Average Pooling

- It retains information about the lesser essential features.
- It simply downscales by dividing the input matrix into rectangular regions and calculating the average values of each area.



Types of neural networks

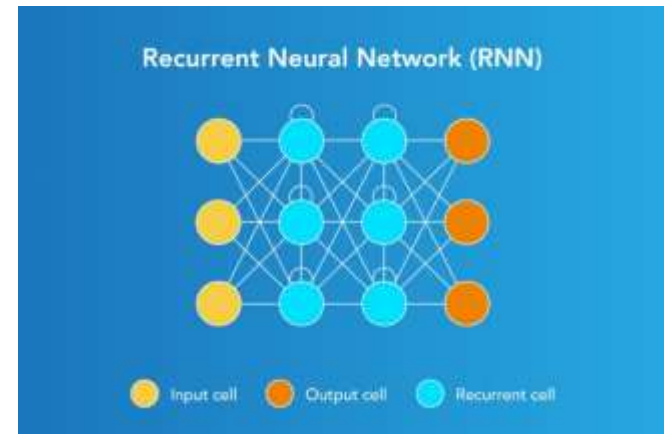
Sum Pooling

It is similar to Max pooling, but instead of calculating the maximum value, we calculate the mean of each sub-region.

Flatten:

Transforming all pooled feature map matrix into a single column which is then fed to the neural network for processing.

- Recurrent Neural Network (RNN) architecture is specifically designed to retain the output of a layer by feeding it back to the input, aiding in predicting the outcome of that layer.
- Typically, the first layer in an RNN is a feed-forward neural network, followed by a recurrent neural network layer where information from previous time-steps is stored using a memory function.
- This implementation involves forward propagation, enabling the network to store information for future use.
- During training, if the prediction is incorrect, a learning rate is applied to make incremental adjustments.



2.8 Recurrent Neural Networks

- This gradual refinement helps the network approach more accurate predictions during the subsequent backpropagation steps.
- By iteratively updating the weights and biases based on the prediction errors, the RNN improves its ability to make correct predictions over time.

Advantages of Recurrent Neural Networks

1. Model sequential data where each sample can be assumed to be dependent on historical ones is one of the advantage.
2. Used with convolution layers to extend the pixel effectiveness.

Disadvantages of Recurrent Neural Networks

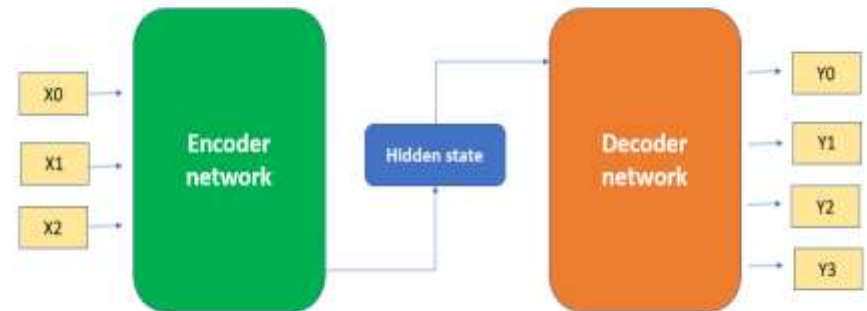
1. Gradient vanishing and exploding problems
2. Training recurrent neural nets could be a difficult task
3. Difficult to process long sequential data using ReLU as an activation function.

Applications of Recurrent Neural Networks

1. Text processing like auto suggest, grammar checks, etc.
2. Text to speech processing
3. Image tagger
4. Sentiment Analysis
5. Translation

4. Encoder Decoder Networks

- The encoder-decoder architecture, widely utilized in various natural language processing and computer vision applications, is a deep learning framework comprising two essential elements: an encoder and a decoder.
- In this architecture, the encoder accepts an input sequence and produces a fixed-length vector representation, often referred to as a hidden or "latent representation," which condenses the significant information of the input sequence.
- The decoder then employs this latent representation to generate an output sequence.
- Neural networks form the fundamental building blocks or components of the encoder-decoder architecture, allowing for the utilization of various types such as RNNs, LSTMs, and CNNs, depending on the specific encoder-decoder design.



2.8 Encoder Decoder Neural Networks

5. Recursive Neural Network

- Recursive Neural Networks (RvNNs) are a class of deep neural networks that can learn detailed and structured information.
- The word recursive indicates that the neural network is applied to its output.
- Recursive Neural Networks (RvNNs) are a type of deep neural network architecture that can effectively process and analyze hierarchical or tree-like structured data. They are particularly useful for tasks such as parsing sentences or working with tree-structured inputs.
- The key idea behind RvNNs is the recursive application of the same set of weights on structured inputs.
- This recursive process involves combining child nodes to generate parent nodes in a hierarchical manner.
- Each child-parent connection in the tree structure is associated with a weight matrix, and similar child nodes share the same weights. This shared weight scheme allows the RvNN to leverage the structural information present in the data.

Recursive Neural Network

- By recursively applying the weights and combining child nodes, an RvNN can learn detailed and structured information from the input data.
- This capability makes RvNNs well-suited for tasks that involve analyzing and extracting hierarchical relationships.
- Parsing sentences is a common example where RvNNs are used, as they can effectively capture the syntactic structure of a sentence by recursively combining word representations.
- In summary, Recursive Neural Networks (RvNNs) are a class of deep neural networks that excel at handling hierarchical data.
- By recursively applying the same set of weights on structured inputs, RvNNs can learn and extract structured information. They are particularly useful for tasks like sentence parsing, where the entire sentence needs to be parsed and analyzed.

Recursive Neural Network Implementation

- A Recursive Neural Network (RvNN) is employed for analyzing the sentiment in natural language sentences, which is a crucial task in Natural Language Processing (NLP). It aims to identify the writer's tone and sentiments expressed in a specific sentence.
- By recognizing basic labels related to the writing tone, it helps in determining whether the sentence reflects a constructive writing style or includes negative word choices.
- Additionally, it seeks to identify smaller components such as nouns or verb phrases and organize them in a syntactic hierarchy.
- During the traversal of nodes, a variable called 'score' is computed, providing information about the pair of phrases and words that should be combined to form the optimal syntactic tree for a given sentence.

- To illustrate, let's consider the phrase "a lot of fun" in the sentence "Programming is a lot of fun." An RNN representation of this phrase would be inadequate because it considers only sequential relationships.
- The state of the RNN varies with the representation of preceding words, making it unable to represent subsequences that do not occur at the beginning of the sentence.
- In contrast, a Recursive Neural Network (RvNN) with its hierarchical architecture can effectively store the representation of the specific phrase, residing in the hidden state of the node $R_{\{a\ lot\ of\ fun\}}$. Thus, Recursive Neural Networks enable complete implementation of syntactic parsing.

Advantages:

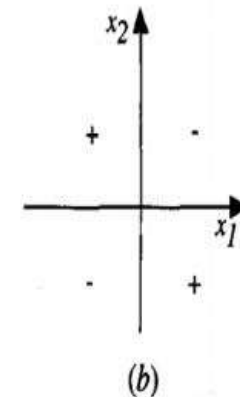
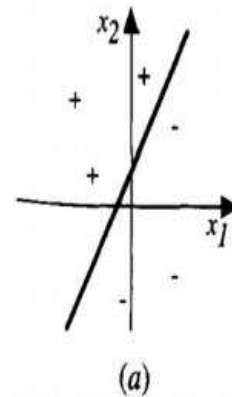
- The two significant advantages of Recursive Neural Networks for Natural Language Processing are their structure and reduction in network depth.
- As already explained, the tree structure of Recursive Neural Networks can manage hierarchical data like in parsing problems.
- Another benefit of RvNN is that the trees can have a logarithmic height. When there are $O(n)$ input words, a Recursive Neural Network can represent a binary tree with height $O(\log n)$. This lessens the distance between the first and last input elements. Hence, the long-term dependency turns shorter and easier to grab.

Disadvantages of RvNNs for Natural Language Processing

- The main disadvantage of recursive neural networks can be the tree structure. Using the tree structure indicates introducing a unique inductive bias to our model. The bias corresponds to the assumption that the data follow a tree hierarchy structure. But that is not the truth. Thus, the network may not be able to learn the existing patterns.
- Another disadvantage of the Recursive Neural Network is that sentence parsing can be slow and ambiguous. Interestingly, there can be many parse trees for a single sentence.
- Also, it is more time-consuming and labor-intensive to label the training data for recursive neural networks than to construct recurrent neural networks. Manually parsing a sentence into short components is more time-consuming and tedious than assigning a label to a sentence.

6. Multilayer Neural Networks

- A single-layer neural network will work only for linearly separable data and not for non-linearly separable data. Hence there is a need for Multilayer Neural Networks, to be able to work with non-linearly separable data.
- A multi-layer Neural Network has two hidden layers. Hidden layers, whose neurons are not directly linked to the output, are used in multilayer networks to address the classification issue for non-linear data.
- The hidden layers can be understood geometrically as extra hyper-planes that increase the network's separation capability. Typical multilayer network designs are seen in the Figure 2.9.



2.9 Figure (a) -> Training Set is Linearly Separable

2.9 Figure (b) -> Training Set is non-linearly Separable

6. Multilayer Neural Networks

This new design raises a new challenge: how to train concealed units whose expected output is unknown. This problem can be solved using the Backpropagation technique.

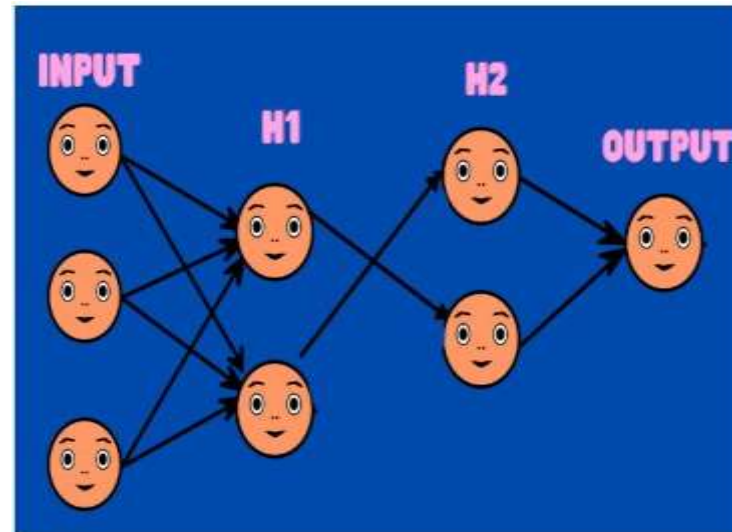


Fig. 2.10 Multilayer Neural Networks

Convolutional neural networks (CNN)	Recurrent neural networks(RNN)
In deep learning, a convolutional neural network (CNN, or ConvNet) is a class of deep neural networks, most commonly applied to analyzing visual imagery.	A recurrent neural network (RNN) is a class of artificial neural networks where connections between nodes form a directed graph along a temporal sequence.
It is suitable for spatial data like images.	RNN is used for temporal data, also called sequential data.
CNN is a type of feed-forward artificial neural network with variations of multilayer perceptron's designed to use minimal amounts of preprocessing.	RNN, unlike feed-forward neural networks- can use their internal memory to process arbitrary sequences of inputs.
This CNN takes inputs of fixed sizes and generates fixed size outputs.	RNN can handle arbitrary input/output lengths.
CNN's are ideal for images and video processing.	RNNs are ideal for text and speech analysis.
Applications include Image Recognition, Image Classification, Medical Image Analysis, Face Detection and Computer Vision.	Applications include Text Translation, Natural Language Processing, Language Translation, Sentiment Analysis and Speech Analysis.

Comparison Attribute	Feed-forward Neural Networks	Recurrent Neural Networks
Signal flow direction	Forward only	Bidirectional
Delay introduced	No	Yes
Complexity	Low	High
Neuron independence in the same layer	Yes	No
Speed	High	slow
Commonly used for	Pattern recognition, speech recognition, and character recognition	Language translation, speech-to-text conversion, and robotic control

Total Number of Internal Parameters:

- A general-principle is to have about ten times as many data points as there are the total number of internal parameters.
- The number of internal parameters in a neural network is total number of weights + the total number of biases.
- The total number of weights equals the sum of the products of each pair of adjacent layers.
- The total number of biases is equal to the number of hidden neurons + the number of output neurons.

For example, consider a neural network with 13 input neurons, two hidden layers of 5 and 4 neurons, and an output layer of 3 neurons.

The total number of weights is $(13 * 5) + (5 * 4) + (4 * 3) = 97$.

The total number of biases is $5 + 4 + 3 = 12$.

Which makes the total number of parameters $97 + 12 = 109$.

Stochastic Gradient Descent

- Gradient Descent is a widely employed technique in machine learning projects, involving an iterative optimization process aimed at locating the optimal value (either minimum or maximum) of an objective function. This method effectively adjusts a model's parameters with the goal of minimizing a cost function.
- The main objective of gradient descent is to determine the model parameters that yield optimal accuracy on both training and test datasets.
- In gradient descent, the gradient refers to a vector that indicates the general direction of the steepest ascent of the function at a specific point.
- By moving in the opposite direction of the gradient, the algorithm gradually descends towards lower values of the function, ultimately reaching its minimum.
- Stochastic Gradient Descent (SGD) differs from traditional Gradient Descent by selecting only a single random training example or a small batch for each iteration to compute the gradient and update the model parameters. This random selection introduces randomness into the optimization process, which is why it is referred to as "stochastic" Gradient Descent.

- The main advantage of SGD is its **computational efficiency**, particularly when working with large datasets. By utilizing a single example or a small batch, the computational cost per iteration is greatly reduced compared to conventional Gradient Descent methods that necessitate processing the entire dataset.
- **Stochastic Gradient Descent Algorithm:**
- **Initialization:** Randomly assign initial values to the model's parameters.
- **Set Parameters:** Determine the number of iterations and the learning rate (α) for parameter updates.
- **Stochastic Gradient Descent Loop:** Repeat the following steps until either the model converges or the maximum number of iterations is reached:
 - a. Introduce randomness by shuffling the training dataset.
 - b. Iterate over each training example (or a small batch) in the shuffled order.
 - c. Calculate the gradient of the cost function with respect to the model parameters using the current training example (or batch).

- d. Update the model parameters by taking a step in the direction opposite to the gradient, scaled by the learning rate.
- e. Evaluate the convergence criteria, such as comparing the cost function between consecutive gradient iterations. Return Optimized Parameters: Once the convergence criteria are satisfied or the maximum iterations are reached, return the optimized model parameters.

In SGD, due to selecting only one random sample from the dataset per iteration, the algorithm's path to the minimum is typically noisier compared to standard Gradient Descent. However, the precise path taken by the algorithm is less significant as long as it reaches the minimum, and it achieves this goal with significantly reduced training time.

Advantages of Stochastic Gradient Descent:

1. **Speed:** SGD is faster compared to other variants of Gradient Descent like Batch Gradient Descent and Mini-Batch Gradient Descent because it updates the parameters using only one example at a time.
2. **Memory Efficiency:** SGD is memory-efficient as it updates the parameters for each training example individually, making it suitable for handling large datasets that cannot fit into memory.
3. **Avoidance of Local Minima:** SGD has the ability to escape local minima due to its noisy updates, enabling it to converge towards a global minimum.

Disadvantages of Stochastic Gradient Descent:

- 1.Noisy Updates: The updates in SGD are noisy and have high variance, which can introduce instability into the optimization process and cause oscillations around the minimum.
- 2.Slow Convergence: SGD may require more iterations to converge to the minimum since it updates the parameters for each training example individually.
- 3.Sensitivity to Learning Rate: The choice of learning rate is critical in SGD. A high learning rate can cause the algorithm to overshoot the minimum, while a low learning rate can lead to slow convergence.
- 4.Less Accuracy: Due to the noisy updates, SGD may not converge to the exact global minimum and can yield a suboptimal solution. This can be mitigated by employing techniques such as learning rate scheduling and momentum-based updates.

Backpropagation Algorithm

- Backpropagation, an essential concept in neural networks, plays a crucial role in optimizing data classification.
- To achieve this, we need to update the weights of the parameters and biases. However, performing this task in a deep neural network poses a challenge. In linear regression, we employ gradient descent to optimize the parameters. Similarly, in deep neural networks, we utilize the Backpropagation algorithm along with gradient descent.
- For a single training example, the Backpropagation algorithm computes the gradient of the error function. It can be expressed as a function within the neural network. Backpropagation encompasses a collection of methods that efficiently train artificial neural networks by leveraging a gradient descent approach, which makes use of the chain rule.
- The key characteristics of Backpropagation are its iterative, recursive, and efficient nature. It calculates updated weights to enhance the network's performance until it can successfully execute the designated task it is being trained for. However, it is necessary to have knowledge of the derivatives of the activation function during the network design phase for Backpropagation to work effectively.

Backpropagation Algorithm

- To gain a deeper understanding of how Backpropagation updates weights, let's begin with an illustrative example and examine the mathematical process involved.

Input values

$x_1=0.05$ $x_2=0.10$

Initial weight

$w_1=0.15$	$w_5=0.40$
$w_2=0.20$	$w_6=0.45$
$w_3=0.25$	$w_7=0.50$
$w_4=0.30$	$w_8=0.55$

Bias Values

$b_1=0.35$ $b_2=0.60$

Target Values

$T_1=0.01$ $T_2=0.99$

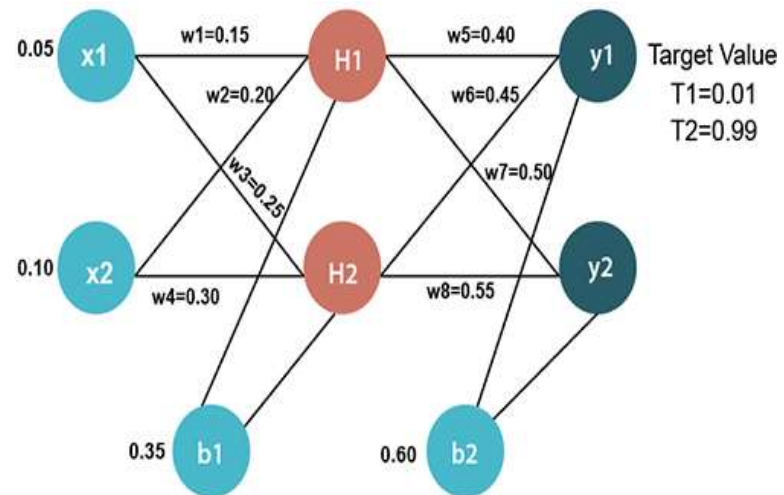


Figure 2.11 Backpropagation example

Backpropagation Algorithm

Now, we first calculate the values of H1 and H2 by a forward pass.

Forward Pass

To find the value of H1 we first multiply the input value from the weights as

$$H1 = x1 \times w1 + x2 \times w2 + b1$$

$$H1 = 0.05 \times 0.15 + 0.10 \times 0.20 + 0.35$$

$$H1 = 0.3775$$

To calculate the final result of H1, we performed the sigmoid function as

$$H1_{final} = \frac{1}{1 + \frac{1}{e^{H1}}}$$

$$H1_{final} = \frac{1}{1 + \frac{1}{e^{0.3775}}}$$

$$H1_{final} = 0.593269992$$

Backpropagation Algorithm

We will calculate the value of H2 in the same way as H1

$$H2 = x1 \times w3 + x2 \times w4 + b1$$

$$H2 = 0.05 \times 0.25 + 0.10 \times 0.30 + 0.35$$

$$H2 = 0.3925$$

To calculate the final result of H1, we performed the sigmoid function as

$$H2_{\text{final}} = \frac{1}{1 + \frac{1}{e^{H2}}}$$

$$H2_{\text{final}} = \frac{1}{1 + \frac{1}{e^{0.3925}}}$$

$$\mathbf{H2_{\text{final}} = 0.596884378}$$

Backpropagation Algorithm

Now, we calculate the values of y_1 and y_2 in the same way as we calculate the H_1 and H_2 .

To find the value of y_1 , we first multiply the input value i.e., the outcome of H_1 and H_2 from the weights as $y_1 = H_1 \times w_5 + H_2 \times w_6 + b_2$

$$y_1 = 0.593269992 \times 0.40 + 0.596884378 \times 0.45 + 0.60$$

$$y_1 = 1.10590597$$

To calculate the final result of y_1 we performed the sigmoid function as

$$y_{1_{\text{final}}} = \frac{1}{1 + \frac{1}{e^{y_1}}}$$

$$y_{1_{\text{final}}} = \frac{1}{1 + \frac{1}{e^{1.10590597}}}$$

$$y_{1_{\text{final}}} = 0.75136507$$

Backpropagation Algorithm

We will calculate the value of y2 in the same way as y1
 $y2 = H1 \times w7 + H2 \times w8 + b2$

$$y2 = 0.593269992 \times 0.50 + 0.596884378 \times 0.55 + 0.60$$
$$y2 = 1.2249214$$

To calculate the final result of H1, we performed the sigmoid function as

$$y2_{final} = \frac{1}{1 + \frac{1}{e^{y2}}}$$
$$y2_{final} = \frac{1}{1 + \frac{1}{e^{1.2249214}}}$$
$$y2_{final} = 0.772928465$$

Our target values are 0.01 and 0.99. Our y1 and y2 value is not matched with our target values T1 and T2.

Backpropagation Algorithm

Now, we will find the total error, which is simply the difference between the outputs from the target outputs. The total error is calculated as

$$\begin{aligned} E_{\text{total}} &= \sum \frac{1}{2} (\text{target} - \text{output})^2 \\ &= \frac{1}{2} (t_1 - y_{1\text{final}})^2 + \frac{1}{2} (T_2 - y_{2\text{final}})^2 \\ &= \frac{1}{2} (0.01 - 0.75136507)^2 + \frac{1}{2} (0.99 - 0.772928465)^2 \\ &= 0.274811084 + 0.0235600257 \\ E_{\text{total}} &= \mathbf{0.29837111} \end{aligned}$$

Now, we will backpropagate this error to update the weights using a backward pass.

Basic Terms

Epoch:

- The neural network learns the patterns of input data by reading the input dataset and applying different calculations on it.
- However, the neural network doesn't make it only once, it learns, again and again, using the input dataset and learned results from the previous trials.
- Each trail to learn from the input dataset is called an epoch.
- So an epoch refers to one cycle through the full training dataset.

Iteration:

For each complete epoch, we have several iterations. Iteration is the number of batches or steps through partitioned packets of the training data, needed to complete one epoch.

Batch:

Batch is the number of training samples or examples in one iteration. The higher the batch size, the more memory space we need.

Examples

Example 1:

- If we have a training set of 1 million images in total. it's a big dataset to feed them all at a time to the network. While training the network, the size of data is keeping to increase in memory. So we'll divide the dataset into parts or batches.
- If we set the batch size to 50K, this means the network needs 20 ($1M/50K$) iterations to complete one epoch.

Example 2:

- Let's say we have 2000 training examples that we are going to use .
- We can divide the dataset of 2000 examples into batches of 500 then it will take 4 iterations to complete 1 epoch.

References

- [1] <https://www.javatpoint.com/deep-learning>
- [2] <https://www.sciencedirect.com/science/article/abs/pii/S1047320316302267>

Thank You...

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

